

Strings and Numbers in JS

Lec-3

Strings in JS

- Strings are for storing text.
- Strings are written within single quotes or double quotes.
- **Eg:**

```
let carName = "Ferrari 250";  
let carName = 'Ferrari 250';
```

Note:

In order to find length of any string we use **length property** along with the use of '**dot(.) operator**'.

Eg:

```
let name = 'Jimmy'  
console.log(name.length) // Output: 5
```

String Concatenation

- String concatenation is a **way to join one string with another string or to join a string with data of some other type**.
- **Eg:**

```
let age = 20  
let stmt = "My age is: "  
console.log(stmt + age); // output: My age is 20
```

Note:

Concatenating Strings with numbers or with any other datatype in such way is not a good practise in writing your JS code.

Template Literals

- Template literals use **backticks (` `)** to define a string rather than **single/double quotes (' ' / " ")**.
- **Eg:**

```
let instrument = `Guitar`;
```

String Interpolation

- **Template Strings** allow variables in strings.
- Template strings provide an easy way to **interpolate** variables in strings.
- **Eg:**

```
let currentAge = 16  
console.log(`My Current age is: ${currentAge}`); // output: My current age is 16
```

String Methods

- String Methods are used to **perform manipulation with Strings**.
- Since strings in JS are **primitive** and **mutable** all the string methods whenever make any changes, produces a new string.
- These methods **never change the original string**.

toUpperCase() : Converts a String into an upper case.

```
let gameName = 'thanos'
console.log(gameName.toUpperCase()); // output: THANOS
```

toLowerCase() : Converts a String into an lower case.

```
let gameName = 'THANOS'
console.log(gameName.toLowerCase()); // output: thanos
```

charAt() : returns the character at a specified index (position) in a string.

```
let gameName = 'thanos'
console.log(gameName.charAt('2')); // output: a
```

indexOf() : Searches a string for a specific value and returns the index (position) of its first occurrence.

```
let gameName = 'thanos'
console.log(gameName.indexOf('n')); // output: 3
```

slice() : Extracts a part of a string and returns the extracted part in a new string. The method takes 2 parameters: start position, and end position (**end not included**).

```
let gameName = 'thanos'
console.log(gameName.slice(0,4)); // output : than
```

Note:

slice() allows negative values as parameters. If a parameter is negative, the position is counted from the end of the string.

```
let gameName = 'thanos'
console.log(gameName.slice(0,-4)); // output : th
```

substring() : This method is similar to **slice()**, the difference is that negative values as parameters are ignored by this method and even after giving negative parameters it starts from *0th* position only.

```
let gameName = 'thanos'
console.log(gameName.substring(1,4)); // output : han
```

trim() : removes whitespace from both sides of a string.

```
let anotherName = "  thanos  "
console.log(anotherName); // output:  thanos
console.log(anotherName.trim()); // output: thanos
```

replace() : replaces a specified value with another value in a string. This method by default replaces **only the first** match. This method is also a **case sensitive** method.

```
let RandomSentence = 'Use Claude, Gemini or Gemini'
console.log(RandomSentence.replace('Gemini', 'ChatGPT'));
// output: Use Claude, ChatGPT or Gemini
```

Note:

In order to replace all matches we use a **regular expression** with **'/g' flag**.

```
let RandomSentence = 'Use Claude, Gemini or Gemini'
console.log(RandomSentence.replace(/Gemini/g, 'ChatGPT'));
// output: Use Claude, ChatGPT or ChatGPT
```

In order to replace all matches **irrespective of the case** we use a regular expression with **'/i' flag**.

```
let ideTool = 'I Love ANTIGRAVITY'
console.log(ideTool);
// output: I Love ANTIGRAVITY
console.log(ideTool.replace(/antigravity/i,'VSCode'));
// output: I Love VSCode
```

includes() : This method is used to **determine whether a string contains a specific substring**. It returns a **boolean value (true or false)**.

```
let url = "https://globalSearch.com/Search"
console.log(url.includes('global')); // Output: true
```

Numbers in JS

- JavaScript has **only one type** of number.
- Numbers can be written **with** or **without decimals**.
- JavaScript numbers are always **64-bit** floating point.

Number Methods in JS

toString() : Returns a number as a string.

```
const aNum = 65;
console.log(aNum); // Output: 65
console.log(typeof aNum); // Output: number

console.log(typeof aNum.toString()); // Output: string
```

Note:

toString() method can also take a optional argument to convert a number to a different base.

```
let aNum = 15
console.log(aNum.toString(2)) // Output: 1111
```

toFixed() : Returns a number as a string, with the number written with a specified number of decimals.

```
const aNum = 3.141592

console.log(aNum.toFixed(2)) // Output: 3.14

console.log(aNum.toFixed(5)) // Output: 3.14159

console.log(typeof aNum) // Output: number

console.log(typeof aNum.toFixed(3)) // Output: string
```

toPrecision() : Returns a number as a string, with a number written with a specified length.

```
let aNum = 9.56563
console.log(aNum.toPrecision(2)) // Output: 9.6
console.log(aNum.toPrecision(3)) // Output: 9.57
console.log(aNum.toPrecision(4)) // Output: 9.566
console.log(aNum.toPrecision(10)) // Output: 9.565630000
```

Note:

toFixed() starts **counting after the decimal**, whereas **toPrecision()** **counts all meaningful digits** in entire number.

toLocaleString() : This is the primary method for making data human-readable based on a user's language and region.

```
const aAmnt = 744000

console.log(aAmnt.toLocaleString()) // Ouput: 744,000

console.log(aAmnt.toLocaleString('en-IN')) // Output: 7,44,000
```


JavaScript Math Object

- The JavaScript Math object allows you to perform mathematical tasks
- The Math object is static.
- The Syntax to Math any number is:

```
Math.method(number)
```

.round() : Returns a nearest integer.

```
const numb = 69.14  
console.log(Math.round(numb)) // Output: 69
```

.ceil() : Returns a rounded up nearest integer.

```
const numb = 69.14  
console.log(Math.ceil(numb)) // Output: 70
```

.floor() : Returns a rounded down nearest integer.

```
const numb = 69.14  
console.log(Math.floor(numb)) // Output: 69
```

.pow(x,y) : Returns the value of x to the power of y.

```
console.log(Math.pow(2,4)) // Output: 16
```

.sqrt() : Returns the square root of a value.

```
console.log(Math.sqrt(144)) // Output: 12
```

.abs() : Always returns the absolute (positive) value.

```
console.log(Math.abs(-63)) // Output: 63
```

.min() : Used to find lowest value in the list of arguments.

```
console.log(Math.min(53,85,96,2,15)) // Output: 2
```

.max() : Used to find highest value in the list of arguments.

```
console.log(Math.max(53,85,96,2,15)) // Output: 96
```

.random() : returns a random number between 0 (inclusive), and 1 (exclusive). It always returns a number lower than 1.

```
console.log(Math.random()) // Output: 0.8923472168130379
```

Note:

Suppose we have to generate a **random number between 1 to 10**.

```
console.log(Math.round(Math.random()*10)+1)
```

Suppose we have to generate a **random number between a given range**.

```
const min = 25  
const max = 50  
  
console.log(Math.round(Math.random() * (max - min + 1) + min));
```